ParallelBayes：时间并行 MCMC 的可核验实现与成本评估

中文研究稿：作者与单位待定

2026 年 10 月

摘要

时间并行 MCMC 同时求解多个采样位置，尝试缩短单条轨迹的计算时间。本文介绍带 R 接口的研究软件 ParallelBayes，将目标、转移核和执行器分开，在固定实际随机输入下比较顺序执行、窗口 quasi-DEER MALA 与 Online Picard RWM。九个目标、两档预算的 CPU/CUDA 研究中，3888 项主任务有 3741 项通过数值标准，42 项数值失败和 105 项进程失败均保留。普通工作流的组内配对几何平均速度比在 CPU 上为 Picard 0.424–1.178，在 CUDA 上为 0.953–1.659；quasi-DEER 的全部组均慢于对应顺序实现。窗口对照显示，批处理机会可被重复求值和较慢的前缀推进抵消。水井案例进一步显示，函数误差较小的配置并不一定具有最低的调用成本；双峰目标的合并符号比例则可由均衡初值维持，而非来自跨峰探索。对选定 NUTS 工作流的有限定位还观察到完整采样后的诊断内存停止。独立原数组重建、完整失败记录与 R 调用测量共同表明，时间并行执行收益需要结合数值输出标准、具体函数和工作流边界评价。

1 引言

MCMC 通过构造马尔可夫链近似后验期望。常规实现按顺序生成样本，下一次更新依赖当前状态。多链并行同时运行多条轨迹；时间并行则尝试同时求解一段轨迹，以利用 CPU 或 GPU 的并行计算能力。Newton 类路径求解与增量 Picard 提供了两条具体路线 [1, 2]，但并行求值还会引入重复工作、导数近似和同步，其局部执行收益是否保留到后验推断，需要区别考察。

本文围绕两个问题展开：固定核参数、初值及实际随机输入时，时间并行执行能否更快完成同一递推；允许不同核采用各自配置后，所选后验函数的误差与工作流成本如何变化。第一类比较控制轨迹的统计信息，第二类同时受初始化、调参和混合影响。本文实现 ParallelBayes，为这两类比较提供可复用目标接口、独立路径参考、实际输入与失败记录，并用指定 CPU/CUDA 条件下的实验解释成本差异。

模型访问与推断组件复用 Stan、BridgeStan、BlackJAX 和 Pyro[3, 4, 5, 6]。compareMCMCs、PPL Bench 与 posteriordb 已有跨引擎运行、基准或参考目标支持 [7, 8, 9]；

ParallelMCMC.jl 和 BayesForge 分别覆盖时间并行 MALA 接口与 R/JAX 建模路线[11, 12]。本文增加的是同一实际输入下的执行比较和数值核验流程，以及这些流程揭示的成本与探索差异，不以统一 R 接口本身作为创新依据。研究限于固定维度连续参数、两类 MH 时间并行执行和指定的 Pyro/BlackJAX NUTS 工作流。

2 方法与软件设计

2.1 目标与坐标

设 T 为相应自由坐标间的可微一一变换，原参数为 $\theta = T(q)$，计算使用无约束坐标 $q \in \mathbb{R}^d$，目标为

$$\ell(q) = \log p(y, T(q)) + \log |\det J_T(q)|. \tag{1}$$

模型记录参数顺序、约束映射、完整数据、实现版本与指纹。两个提供方式可省略不同的参数无关常数，因此以 $\ell(q) - \ell(q_0)$、同坐标梯度及变换结果核对目标。正值测试例使用 $\theta = \exp(q)$ 和 lognormal 密度，其 $-\log\theta$ 项与 Jacobian 抵消。这一例同时用于检测遗漏 Jacobian 的实现错误。

R 是建模控制与分析入口。BridgeStan 在 CPU 上调用编译后的 C++ 密度和自动微分，Python 串行循环组织转移；原生 JAX 模型进入 JIT 和批处理；独立 torch 后端使用 NumPy 参考及显式 CPU/CUDA 执行，当前时间并行执行器仍含 Python 主机控制。Stan 模型指针不跨线程共享，Stan 内部线程关闭。此时高频调用边界的成本属于软件工作流，不能从跨后端差异直接推断并行算法的贡献。

目标入口与实测设备	顺序 RWM/MALA	Picard RWM / quasi-DEER MALA	NUTS
Stan/BridgeStan，Mac CPU	支持	不支持	不支持
明确 JAX 目标，CPU	支持	支持对应核	BlackJAX 顺序核
明确 torch 目标，Windows CPU	已核验	已核验，主机控制	独立研究 CLI：Pyro 四链
明确 torch 目标，Windows CUDA	已核验	已核验，主机控制	未接入/核验

表 1: 当前代码的支持范围与实测边界。CPU Pyro 基线是独立研究 CLI，不是 `pb_sample()` 的通用 torch NUTS 选项；四链并发也不是 NUTS 轨迹时间并行。原生 Windows Stan 编译尚未核验。支持数值输出不等于已证实后验精度。

R 函数 `pb_capabilities()` 查询包接口支持的组合；独立研究 CLI 另列运行契约，不能从表1推定全部路线都能由一个 R 函数调用。`pb_benchmark()` 只组织配置列表及 R 结果，正式协议冻结、检查点和完整证据管理依赖 Python CLI 及项目文件。Stan 模

型没有默认独立轨迹参考，`audit=TRUE` 明确拒绝。扩展示例通过公开 Python `Model` 提供 Poisson 目标及独立 NumPy 参考，R 经 reticulate 调用并转换为 posterior 对象；并非 `pb_model()` 可直接接收任意 R 闭包。

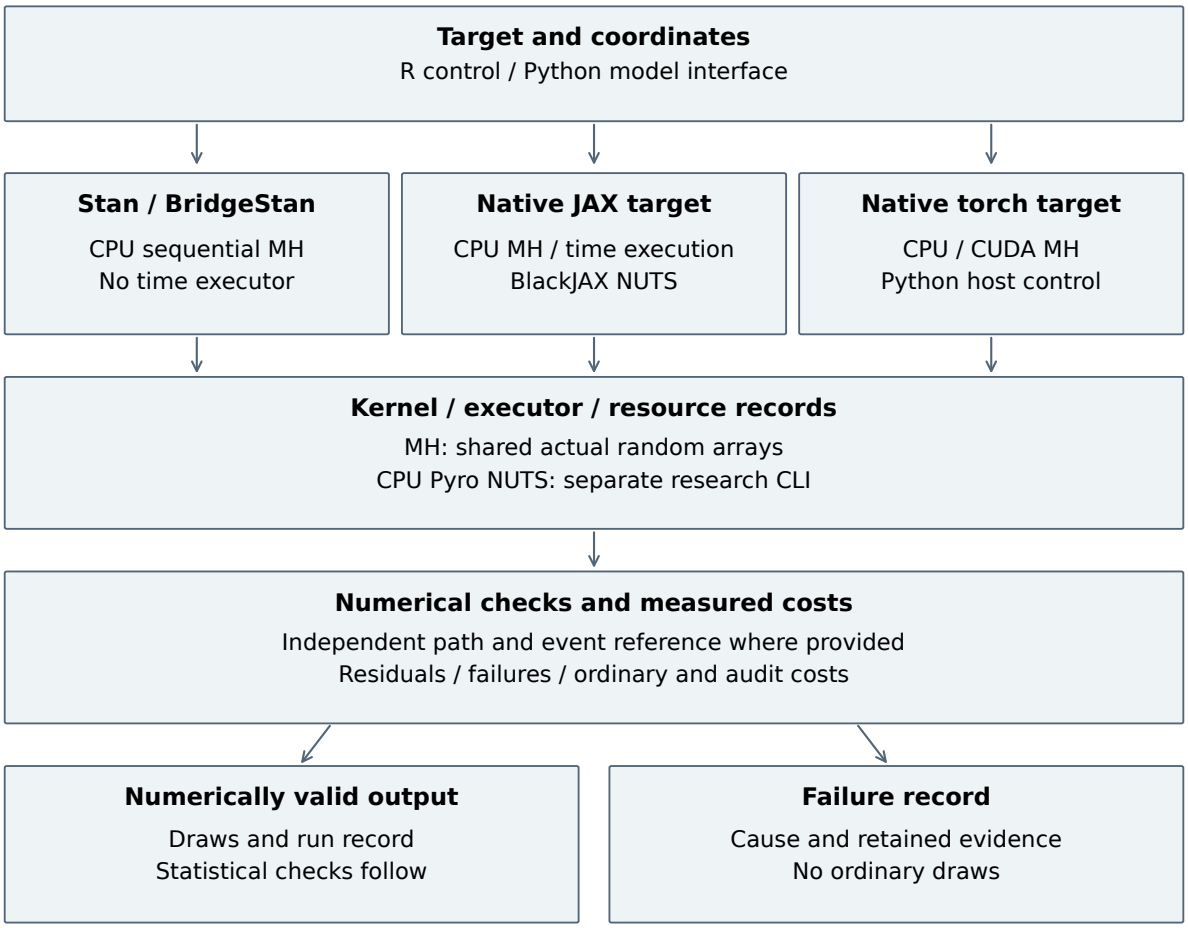


图 1: 当前软件路线与证据流。Stan 只连接 CPU 顺序核；JAX 和 torch 仅接受明确实现的目标。独立顺序参考须另外提供，Stan 入口没有默认参考。NUTS 使用自身随机流及预热，独立 Pyro CLI 另作记录。数值有效输出继续接受统计诊断，失败不能转为正常样本。

2.2 固定随机输入下的核

MALA 的步长约定为

$$q' = q + h\nabla\ell(q) + \sqrt{2h}\, z, \qquad z \sim N(0, I_d). \tag{2}$$

接受对数比包含正反向提议密度：

$$a(q, q') = \ell(q') - \ell(q) - \frac{\|q - q' - h\nabla\ell(q')\|^2 - \|q' - q - h\nabla\ell(q)\|^2}{4h}. \tag{3}$$

当 $\log u < a(q, q')$ 时接受，否则保留 q。随机游走 Metropolis（RWM）使用 $q' = q + sz$，接受对数比为 $\ell(q') - \ell(q)$。拒绝产生的自环计为一个转移。本文的 h 不等于提议噪声标准差；这一约定在所有配对执行中固定。

随机输入由锁定 NumPy 版本下的 Philox 生成，采样与导数探测种子分开。比较固定的是噪声数组和对数均匀随机数组，而非跨软件的同名整数 seed。正式原始输出保存这些数组及其校验和。独立顺序参考使用 NumPy 密度、解析梯度和显式循环，不调用待核验的 JAX 或 torch 转移函数。跨系统重建派生浮点数组不保证逐位一致；重复执行应传递已保存并封存的随机数组，再分别核对文件与数组哈希。

2.3 两类时间并行执行的共同数学对象

普通 MCMC 逐步生成状态。时间并行执行器先猜测一个窗口内的路径，再修正相邻状态的不一致。固定初值和实际随机输入后，每一步都是确定映射，顺序路径满足

$$q_t^\star = F_t(q_{t-1}^\star), \qquad t = 1, \ldots, W. \tag{4}$$

quasi-DEER 用近似导数修正窗口；Online Picard 用接受增量重建路径并提交连续一致的前缀。它们求解的是指定递推，核验比较的也必须是相同初值、核参数及随机数组。

quasi-DEER 的窗口修正。 给定第 k 轮路径，下一轮满足

$$q_t^{(k+1)} = F_t(q_{t-1}^{(k)}) + B_t^{(k)}\{q_{t-1}^{(k+1)} - q_{t-1}^{(k)}\}. \tag{5}$$

旧路径上的映射和矩阵可同时计算；随后把右侧整理为关于新前驱的仿射递推，用结合扫描求解。在精确算术、各次映射有定义、$B_t^{(k)}$ 有限且仿射递推精确求解时，第 k 轮至少恢复前 k 个状态，因而长度 W 的窗口至多需 W 轮。原因是固定初值使第一步正确；若前 k 步已经正确，下一轮保持该前缀，并使第 $k+1$ 步的修正项为零。这是既有因果递推性质 [16, 1]，不依赖 B 必须为真实 Jacobian。它没有为浮点溢出、容差提前停止或一般后验误差提供保证。

硬前向与代理 Jacobian。 令 $b = \mathbf{1}\{\log u < a(q, q')\}$、sg 为停止梯度算子。本实现用

$$g = b + \sigma(a - \log u) - \mathrm{sg}\{\sigma(a - \log u)\} \tag{6}$$

构造代理导数；前向仍返回硬接受/拒绝状态。torch 代码另以停止梯度包裹硬输出，使前向避免软表达式的舍入抵消。这里得到的是代理 Jacobian J_{proxy}，不是硬 MH 映射的真实 Jacobian。每链、每时间位置采用一个实际 Rademacher 向量 v_t，在窗口迭代中复用，并设置

$$B_t^{(k)} = \mathrm{Diag}\Big[\mathrm{clip}\{v_t \odot (J_{\mathrm{proxy,t}}^{(k)} v_t), -1, 1\}\Big]. \tag{7}$$

截断作用于导数近似，不作用于状态。固定探针的复用不支持额外声称每轮自适应矩阵的条件无偏估计。

原论文伪代码先取 $\log\alpha=\min(0,a)$，其平滑门控为 $\sigma(\min(0,a)-\log u)$[1]；本项目归档提交 `5e5b637` 的上游 `samplers.py` 与本实现使用未截断的 a。两种硬接受判据等价，代理导数一般不同，故这里明确报告所测公式。本实现还采用非重叠窗口、重复窗口起点的初猜及硬残差停止，不使用上游的 NaN 归零或状态裁剪；性能比较针对该适配实现。

Online Picard 的增量和确认前缀。 已确认起点为 q_L 时，RWM 的固定输入增量及候选路径为

$$D_t^{(k)}=sz_t\mathbf{1}\left\{\log u_t<\ell(q_{t-1}^{(k)}+sz_t)-\ell(q_{t-1}^{(k)})\right\}, \tag{8}$$

$$q_t^{(k+1)}=q_L+\sum_{i=L+1}^{t}D_i^{(k)}. \tag{9}$$

在新路径的前驱上重算接受事件，只提交连续匹配的前缀。例如四个位置依次为“匹配、匹配、不匹配、匹配”，只能提交前两个。第四个位置的前驱尚未确认。由起点向后归纳，固定噪声下匹配的接受事件确定相同增量，因而确认前缀满足原递推 [2]。torch 批量执行对各链取最短确认长度共同推进，窗口尾部只处理实际存在的转移。浮点前缀和与顺序累加的次序不同，仍须分别检查路径差和接受事件。

2.4 数值输出标准与失败处理

quasi-DEER 停止条件为硬递推残差

$$r=\max_{t,j}\frac{|q_{t,j}-F_t(q_{t-1})_j|}{\mathrm{atol}+\mathrm{rtol}|F_t(q_{t-1})_j|}\le 1. \tag{10}$$

它控制局部一致性，而非长路径误差。独立 NumPy 参考另要求接受事件零失配，且每链无约束路径最大差不超过 $100[\mathrm{atol}+\mathrm{rtol}\max(1,\max|q|)]$，本文协议的两种容差均为 10^{-10}。100 倍系数是指定输出标准，不是误差传播界。全路径尺度会放宽小尺度坐标的阈值；后续逐坐标和函数差只能作为另列的描述，不能追溯改变原合格判据。

迭代耗尽、非有限值或独立审计失配的输出保留原记录，不转换为普通后验样本。当前 MH 实现把非有限提议密度统一视为失败；合法的支持集外提议密度 $-\infty$ 尚未一般地区分，因此有界支持目标的适用范围受限。NUTS 的发散与返回有限数组分别记录。显式顺序回退复用原数组重算整个轨迹并计入失败成本，本文实验不启用回退隐藏求解失败。

2.5 R 用户如何运行和扩展

安装版示例以二维 Poisson 回归为例，$y_i\sim\mathrm{Poisson}(\exp(X_i\beta))$、$\beta_j\sim N(0,4)$。外部 Python 模型提供框架密度、独立 NumPy 密度/解析梯度和恒等参数变换；R 整批调用，再转换为 `posterior` 对象。以下为仓库中实际示例的调用顺序，环境变量在启动 R 前指定，输出使用不存在的新目录。

```
# Once: create the shared data and actual random arrays
python examples/installed_custom_target.py prepare \
  --output output/poisson-inputs
# Set R_LIBS_USER and RETICULATE_PYTHON before starting R
Rscript --vanilla examples/installed-custom-target.R \
  torch output/poisson-inputs output/poisson-fit cpu
```

以下片段展示与上述 R 命令等价的调用，两者选择其一运行。从仓库根目录出发，目录与返回值的关系如下。核和执行器在 Python 示例的 `workflows` 配置中选择，依次为 MALA 顺序/quasi-DEER、RWM 顺序/Picard；主示例对四种组合全部运行。

```
example_dir <- normalizePath("examples", mustWork = TRUE)
inputs <- normalizePath("output/poisson-inputs", mustWork = TRUE)
output <- "output/poisson-fit" # Must not already exist
library(parallelbayes)
pb_environment("torch")
helper <- reticulate::import_from_path(
  "installed_custom_target", path = example_dir, convert = TRUE)
result <- helper$run_example(inputs, output, "torch", "cpu")
print(vapply(result$workflows, function(w) w$status, ""))
for (w in result$workflows) {
  if (!identical(w$status, "completed")) next
  a <- w$draws
  dimnames(a) <- list(NULL, NULL, unlist(w$names))
  draws <- posterior::as_draws_array(a)
  print(dim(draws)) # iteration, chain, parameter
  print(posterior::summarise_draws(draws))
}
```

实际 Mac torch 安装示例返回四个 `completed` 状态，每组样本维度为 `128 × 4 × 2`。第一组顺序 MALA 的 `beta0` 均值为 0.11428、标准差 0.16642、$\hat{R} = 1.01917$，`beta1` 的 $\hat{R} = 1.03180$，说明接口成功返回仍可伴随短链诊断问题。完整脚本同时核对安装模块来源和输入哈希，保存失败项、样本及诊断，并在数值检查失败时返回错误。该例已在 Mac 安装环境下运行 JAX/torch 与 R 入口，四链各 128 次转移用于说明接口；没有预热，不能作为推荐推断预算。它的 Windows/CUDA 组合尚未实测。水井案例另有 Windows CPU/CUDA/R 接口证据。`pb_model()` 不接收任意 R 闭包，Pyro CPU NUTS 也仍是独立研究 CLI。

3 核验方法与实验设计

3.1 实验身份、目标与独立单位

各批次回答不同问题，不能按任务数合并统计证据（表2）。同核配对共享初值、核参数和保存的随机数组；链、缓存回放、跨设备副本均不增加独立重复数。历史 Mac 为 M4/16 GB、JAX float64；Windows 为 Ryzen 7 9800X3D、RTX 5080 和 torch float64，主机控制与同步成本保留。平台间同时改变实现和硬件，只能比较完整工作流。

证据批次	平台与工作流	重复和保留预算	用途/当前状态
Mac 主实验	0.1.0，五工作流	每格 24 份四链输入；256/2048	1920 项，固定预算比较
Mac 修订/SBC	0.1.1，解析生成模型	64 数据集；五共享工作流	配对解析校准检查
Windows 首轮	0.2.0.dev1，CPU/CUDA 四种 MH	每格 4 份四链输入；128/512	512 项，探索性执行比较
Windows 机制	独立机制协议，CPU/CUDA	每模型 2 份输入；窗口 4/16	192 工作流；机制描述
主要实验（compact 协议）	独立冻结版本；九工作流	每目标 24 份四链输入；1024/4096	3888 主任务；完整原数组复核

表 2: 证据身份与用途。缓存回放和安装/恢复检查不计统计重复；详细源码、环境及协议清单见补充材料与项目索引。

目标库涵盖独立与相关高斯 G1/G2、具有高斯后验的潜在 AR(1) A1、logistic L1/L2、中心化/非中心化漏斗 H1/H2、双峰 M1 和水井 W1。历史两轮不含 W1。H1/H2 以不同坐标采样，原尺度输出通过变换对应；相同数值步长不代表原尺度上的同一核，因此只在每个坐标表示内部配对执行。

3.2 函数误差、参考和诊断

对预先规定的函数 f，记录四链平均估计 $\widehat{\mu}_{a,f}^{(r)}(N)$。令 $\mathcal{V}_{a,f,N}$ 为通过数值标准且函数估计有限的重复集合，成功条件下的逐函数误差为

$$\widehat{E}_{a,f}^{\mathrm{valid}}(N) = \frac{1}{|\mathcal{V}_{a,f,N}|} \sum_{r \in \mathcal{V}_{a,f,N}} \frac{(\widehat{\mu}_{a,f}^{(r)}(N) - \mu_f^\star)^2}{s_f^2}. \tag{11}$$

有解析期望时这是有效输出条件下的重复 MSE 估计；有限 MCMC 或数值求积参考下称参考平方差。集合为空时不定义该点。两方法的配对比较另用共同可用集合，失败率始终使用全部 24 次计划重复；$\hat{R} > 1.01$ 用于描述探索情况，不用于事后筛除轨迹。历史分析还报告函数间最大值，主要实验以逐函数结果为主。所有实际函数定义列于表3；

标准化已包含在部分函数本身，其余函数没有转成相同实际意义的相对误差。当前误差计算的额外 $s_f = 1$。历史阈值 $\epsilon = 0.1$ 即平方误差 $\epsilon^2 = 0.01$，对应函数单位中的 RMSE 0.1；它是比较标尺，不是每种应用都充分的精度要求。误差估计的重复不确定性和参考 MCSE 均另列，不与 ϵ 混用。

目标（函数数）	原尺度函数，所有 $s_f = 1$	期望/参考
G1/G2/A1（各 3）	$z,\ z^2,\ \mathbf{1}(z > 1),\ z = (\theta_1 - m_1)/\sqrt{\Sigma_{11}}$	$0, 1, 1 - \Phi(1)$
L1/L2（各 4）	$\theta_1, \theta_2, \sigma(\theta_1), \mathbf{1}(\theta_1 > 0)$	独立 NUTS 有限参考；L2 事件另有重要性参考
H1/H2（各 4）	$v/3, \tanh(v/3), \mathbf{1}(x_1 > 0), \cos(x_1 e^{-v/2})$	$0, 0, 1/2, e^{-1/2}$
M1（4）	$\theta_1/\sqrt{1+\delta^2}, \mathbf{1}(\theta_1 > 0), \theta_2, \theta_2^2$	$0, 1/2, 0, 1$；δ 为模式位置绝对值
W1（7）	$\alpha, \beta, \alpha^2, \beta^2, \sigma(\alpha), \sigma(\alpha+\beta), \mathbf{1}(\beta > 0)$	区间数值包络及独立有限参考；非解析真值

表 3: 九目标共 36 个函数；W1 两项预测概率对应 0 米和 100 米。最终配置见补充材料第 1 节，逐函数参考与精度见第 2 节。历史 $s_f = 1$ 口径保留，不用正式样本方差重新归一化。

L2 原 NUTS 参考的符号事件在 262144 个相关样本中恒零，不能据零经验方差填入零误差；新增独立重要性参考及其近似 MCSE 另列，不追溯改写原参考。W1 另用球算术、四阶求积余项和矩尾界包络七个函数的期望；这一步与后验可正规化证明分开，细节及区间见补充材料第 2 节。对有限参考的敏感性分析与条件于参考的统计区间分开给出。现代秩正态化/折叠 $\hat{R}$、bulk ESS 与 tail ESS 从完整样本计算 [17, 10]，常量输出保留不可判定；这些诊断不替代真实函数误差。

历史 Mac 采用 24 重复、4000 次逐点 bootstrap；原 Windows 每格只有 4 份输入，其 bootstrap 区间只作探索性描述，完整配对点移入补充材料。主要实验主任务按完整四链重复整体重采样，采用 9999 次逐点 BCa、至少 20 份共同可用重复；退化区间保留未定。缓存每格仅 4 份输入，不计算 BCa。失败率使用全部计划重复，误差/耗时的成功条件统计明确标注；它们不等于包含失败损失的无条件风险。

3.3 调参、预算与三种成本边界

历史零初值和固定 MH 步长定义的是指定工作流。主要实验沿用独立开发试验选定的仿射坐标和步长：每目标/核六个候选，每候选两份四链输入，按两次都合格后的平均 ESJD/d 选择。216 次计划开发运行中 212 次完成、4 次失败；该规则优化移动距离，不证明 ESS 或函数误差最优。最终目标、仿射矩阵来源及各核步长集中列于补充材料表 S1 及第 1 节。G2 与 A1 采用完整协方差白化，采样坐标中的目标为标准高斯，不能将其核差异直接解释为尚存的原尺度相关性。正式 MH 初值在采样坐标中取 $N(0, 4I)$，

M1 首坐标分置两侧；窗口 32，quasi-DEER 每窗口最多 2048 轮，Picard 最多 $N+512$ 轮。CPU Pyro NUTS 使用四进程，每链预热 1024 步、目标接受率 0.8、最大树深 8；MH 舍弃 512 步。开发调参耗时单列，不能隐记为零或假定每次拟合重新支付。

主要实验固定两档保留预算 1024/4096 及 24 重复。受计算资源与存储限制，研究在早期大规模网格启动后调整了实验设计。新实验使用单独的协议标识和随机输入，早期结果另行保留；该过程不属于完全事前预注册。Windows 冻结执行源为 3a37a89，交付为 94bebec。Mac 从完整原件独立重建全部 4144 项主任务与缓存记录，再按冻结统计规则生成结果。缓存只在 G1/G2/L2/W1 各分配 64 个探测，合计 1024 次调用，其余目标没有计划缓存测量。两个已测预算点不用于插值精确达标时间。

计时分别回答三个问题：缓存执行衡量准备后的内核；普通批量工作流包含进程启动、输入/目标、适应或丢弃、采样、变换、输出和 R 诊断；研究审计另含独立顺序参考与完整证据检查。CUDA 边界显式同步，后两种口径为嵌套边界，不相加。历史 Mac 的普通工作流耗时口径只是从已有计时扣除审计的记账估计；后继进程测量才是直接观测。两者仍不含最外层 R 前端、环境安装和全局归档传输。R 前端另以一个既有 Poisson 输入、32 个串行新 R 进程和 64 次技术调用直接测量；普通与审计模式各自记录，不回填正式主任务。误差与时间使用同一组函数可用重复，缺少完整计时的任务不能通过删行产生曲线。

4　实验结果

4.1　路径一致性与统计校准是不同检查

主要实验的 3414 项有效 MH 主任务及 1024 次缓存调用由 Mac 独立重放，共 54448128 次转移，接受事件失配为零；主任务最大路径差为 2.91910×10^{-7}，最大误差/原界限比为 0.884。另有 327 份 NUTS 有效输出通过其各自输出检查规则核对，未将它们称作 MH 同轨迹重放。

Mac 的 1920 项和 Windows 首轮的 512 项均通过各自规定的数值输出检查，独立参考的接受事件零失配；最大路径差分别为 2.36×10^{-8} 和 5.212×10^{-10}。两者容差尺度与实现不同，不能用这两个最大值作算法精度排名。跨系统诊断重建还发现小数读取与中位数末位变化影响秩诊断；改用实际二进制输入后，剩余差异定位于折叠分量，未改变相关任务诊断不良的判断。详细定位及失败记录进入补充材料。

H1 的数值失败进一步显示，路径差与函数估计差需要分别衡量。原 42 份 MALA 失败中 40 份来自 quasi-DEER、2 份来自顺序执行；前者均满足内部局部残差条件。将这些候选及配对有效输出共 80 项与独立 NumPy 路径比较，接受事件和符号函数仍无失配。在剔除原 512 步后的保留段，四个函数的最大逐点绝对差为 3.27×10^{-3}，最大四链合并均值差为 3.08×10^{-7}（补充材料第 4 节）。均值差可受偏差持续时间和符号抵消影响，因而不能代替完整路径标准；42 项原失败继续排除。这些量描述指定浮点路径间的差异，不是对未知后验真值的误差界。

SBC 采用 $\theta \sim N(0,1)$、$y_j \mid \theta \sim N(\theta,1)$，$j=1,\dots,8$，解析后验为 $N(\sum_j y_j/9, 1/9)$。在同一批 64 数据集上，解析 90% 区间覆盖 54 例；MALA 及其时间并行执行也覆盖 54 例，但与解析区间有 4 例覆盖分歧。RWM 及 NUTS 覆盖 53 例。总体比例接近不能说明逐数据集区间相同，五种工作流也不能算五份独立校准证据。端点、后验矩和数据依赖对数似然秩另列；测试量的选择决定 SBC 能发现哪些问题 [18]。

4.2 普通工作流的配对执行成本

主要比较采用普通工作流耗时，覆盖全部九目标、两预算和两个设备上的同核配对（图2）。Picard 的配对几何平均速度比在 CPU 上为 0.424–1.178，18 组中 7 组点估计超过 1；CUDA 上为 0.953–1.659，14 组超过 1。quasi-DEER 在 CPU 和 CUDA 上的范围分别为 0.116–0.394 和 0.234–0.408，全部低于 1。区间展示逐点不确定性，超过 1 的点数不等于已确证加速的组数。

这一普通工作流耗时比较比历史缓存收益更接近一次批量使用，但仍条件于给定核配置和成功输出。历史 Mac 两类时间并行执行的所有模型—预算组缓存中位速度比均小于 1；首轮 Windows CUDA Picard 的范围为 1.097–5.230，quasi-DEER 为 0.164–0.505。历史网格与本轮的调参、预算和计时边界不同，不能把范围的变化归因于单一因素。加入双方共有的独立审计成本又会把耗时比推向 1，因此三种口径分别保留。

4.3 为什么较快的执行未必提供更有用的样本

CUDA 的 A1/RWM、512 步配置中，Picard 缓存执行约快 4.09 倍，但提议全部被拒绝，链停在初值，执行器每轮可确认整个窗口。自环降低了路径求解难度，同时使后验探索失效。这一反例解释了确认前缀与统计效率的区别，不能只按每次确认转移的成本推荐执行器。

窗口扩大还会增加重复工作。Mac 机制实验中，G1 四链 Picard 将窗口 16 增至 64，每轮确认数的组中位数仍约 5.57，而前向映射求值次数由 5.50 升至 20.88 次/输出转移，缓存速度比从 0.402 降至 0.170。G2 单链 quasi-DEER 的平均窗口轮数由 11 升至 22.25，速度比从 0.051 降至 0.027。拒绝自环均计入分母，JVP 另计。

Windows 机制协议中的步长、窗口和链数对照也显示重复求值与同步会增加工作量（补充材料历史机制部分）。各模型只有两份机制输入，操作探针还会扰动执行；这些结果与额外工作导致减速相容，尚不能唯一分离 JVP、扫描和主机控制的因果贡献。

4.4 固定预算误差与探索诊断

增加保留预算并未消除所有目标上的探索困难。G2 和 A1 在每链 4096 次保留转移下，RWM 标准化首参数函数的 24/24 份拟合仍有 $\hat{R} > 1.01$；对应 MALA 拟合均未超过此阈值，两设备相同。两目标都已白化，该对照描述指定维度与步长下的核差异，未单独识别相关结构的影响。图3将这一差异与普通工作流耗时对应。

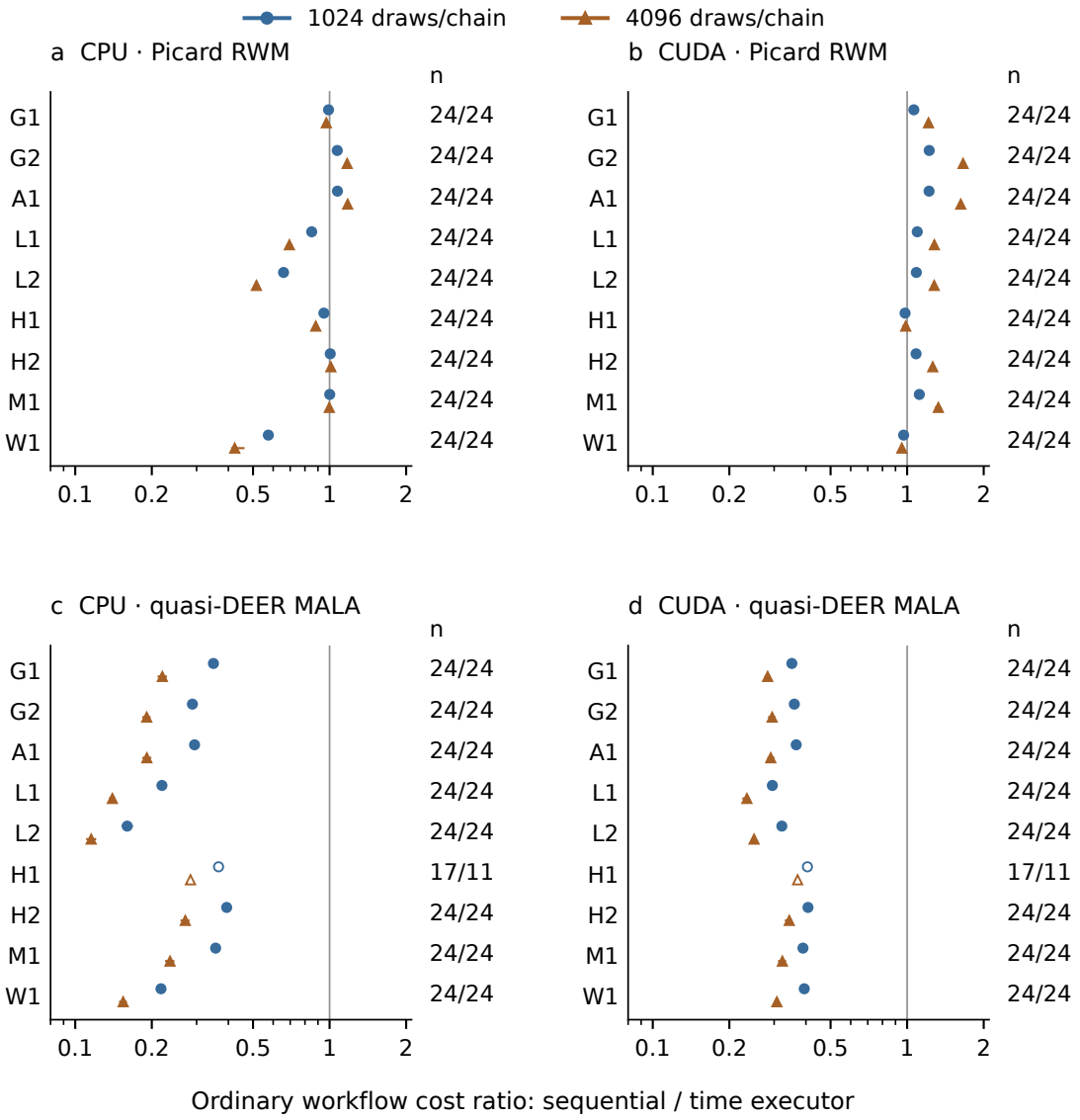


图 2: 全部 72 组普通工作流的同核耗时比，定义为顺序/时间并行执行，比值大于 1 表示时间并行执行较快；点为配对几何平均，线为条件 95% BCa 区间。颜色与形状区分两档保留预算，右侧 n 按 1024/4096 步依次列出共同有效四链重复数。H1/MALA 的四组少于 20 份，按冻结规则只画空心点。部分区间短于点符号。缓存与研究审计口径见完整结果附录图 R1、R5 等各目标的执行耗时图。

在主要实验的 3741 份有效拟合中，所选函数共形成 14981 行诊断，5000 行的有限 $\hat{R} > 1.01$，另有 1290 行未定。函数行共享拟合，不是独立重复。完整结果附录保留九目标 36 函数的两个已测预算点、条件误差区间及相同集合上的耗时；它们没有定义单次可观测停止准则，也没有插值得到精确达标时间。

全部 105 项未分类失败来自 4096 步 CPU Pyro NUTS 四进程工作流：105/216，即 48.6% 的计划拟合未返回完整四链结果；1024 步的 216 项全部有效。长预算各目标失败数依次为 G1 17、G2 0、A1 0、L1 18、L2 18、H1 15、H2 17、M1 20、W1 0，每目标均计划 24 项。部分组仅有 4–9 份有效拟合，成功条件误差和耗时不代表全部尝试。原日志只定位到工作进程异常终止。另有 48 份有效 A1/G2 长预算拟合的 192 条链，在内置 ESS 中各触发约 2 GiB 分配警告，旧诊断缺失；保存样本仍由 R 计算现代诊断。

为区分采样与后处理，另对九个固定旧输入比较 1/4 个 worker 及内置诊断开启/整体旁路。G2/A1 的四 worker 开启条件在每链 4096 个样本完整持久化后进入诊断，随后收到 Job 内存限制消息；旁路条件完成，按预定规则选择的确认调用重复了这一分歧。保存的旧 G2 数组单独进行四 worker ESS 重放也触发 RSS 护栏。Pyro 1.9.2 的旧 ESS 用二维重复数组计算累积最小值，64 维、4096 步时仅该 float64 数组每链即约 2 GiB，与观察到的内存压力相容。六个选定历史失败输入本轮均未复现，因此新证据定位了这些新工作流的诊断阶段故障，仍不重判原 105 项失败；完整条件表、资源计数与独立重建见补充材料的 NUTS 有限定位部分。

双峰目标还显示，正确的合并函数估计不一定来自充分的单链探索。主要实验的 412 份有效 M1 拟合中，411 份保存路径未穿越 $\theta_1 = 0$，其合并正侧比例却恰为 1/2：初值规定两条链在负侧、两条在正侧，等长合并自然保持这一比例。唯一例外是一次 1024 步 CPU NUTS 拟合的一条链穿越零点（图4）。这一现象不否定所有合并估计的准确性；已知模式权重时可另作分层估计。但符号函数的零经验误差不能替代单链探索及不确定性检查。20 份失败拟合另列，不进入有效拟合分母；有效拟合中的退化诊断仍保持未定。

L2 的稀有符号事件说明，绝对误差还取决于函数的概率尺度。原参考与全部 414 份有效拟合均未访问 $\theta_1 > 0$；另立协议的独立重要性参考给出两套估计 1.57×10^{-9} 和 1.76×10^{-9}，各自近似 MCSE 及权重质量见补充材料第 2 节。对应全零估计的事件平方差仅约 10^{-18}，相对于任一正概率参考的相对误差却为 100%。因此，小绝对误差不表示链探索到了事件域。新增参考单列，原 18 项失败、常量函数的未定诊断及全函数判据保持原样。

4.5 水井案例的推断任务

W1 使用 posteriordb 的 3020 个观测，模型为 $\Pr(y_i = 1) = \sigma(\alpha + \beta d_i/100)$，保留原模型对 α, β 的平坦先验。问题是距离与换井概率的关系，以及 0 米、100 米处预测概率的估计成本。两个不同距离处均出现异结局观测，足以证明该二维后验可正规化（补充材料附录）。Stan、NumPy、torch 密度及导数和 Windows 的 CPU/CUDA/R 数组路

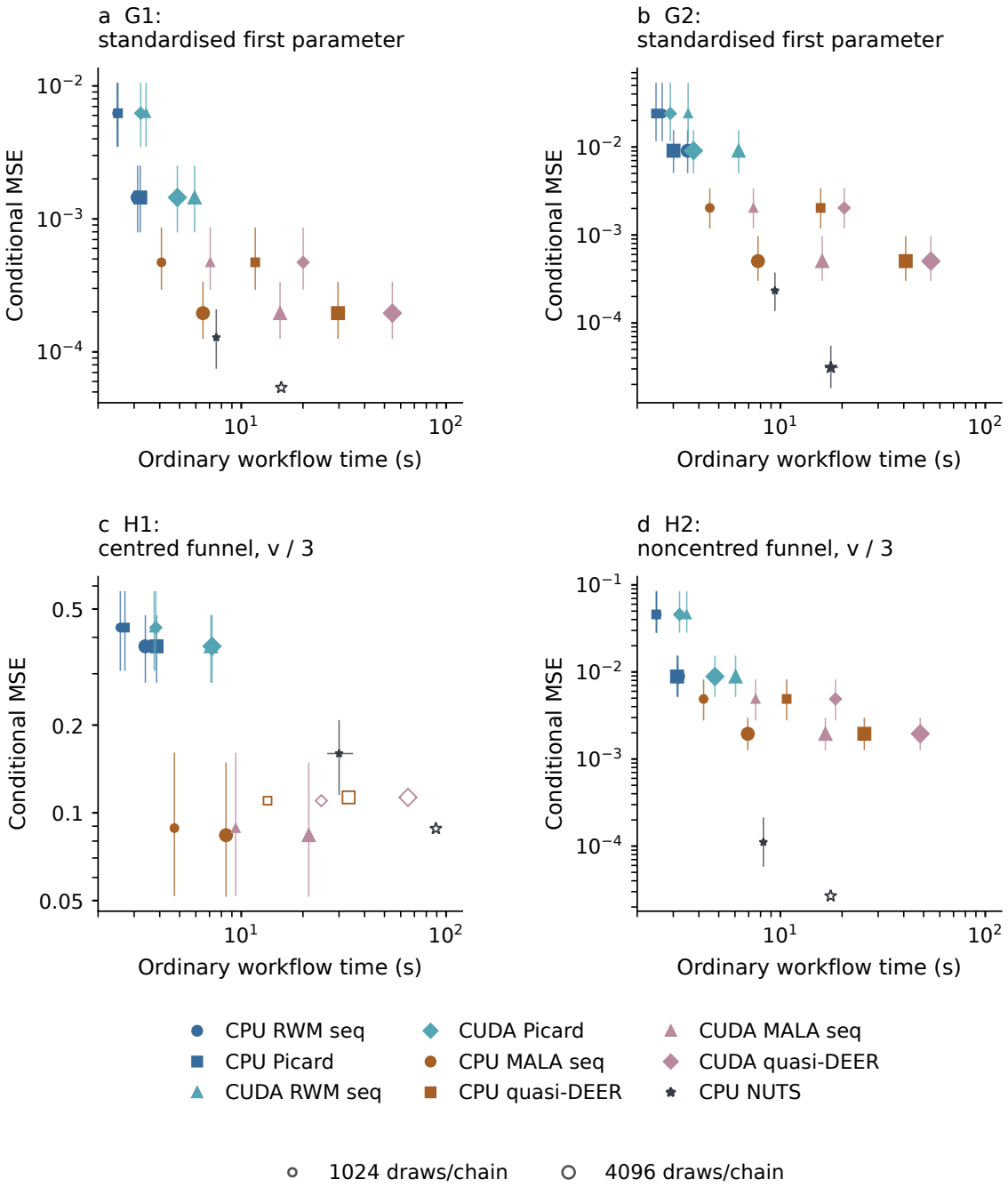


图 3: 已测预算上的函数误差与普通工作流耗时。事后为解释简单目标、白化后维度和漏斗参数化而选择 G1/G2 的标准化首参数均值及 H1/H2 的 $v/3$；四面板均保留全部九工作流、两预算，其他函数见完整结果附录图 R2–R45。点大小区分 1024/4096 预算，横纵线为逐点 95% BCa 区间，空心点表示区间未定。每点计划 24 份四链重复；G1/G2/H2 的 MH 均为 24 份，Pyro NUTS 分别为 24/7、24/24、24/7；H1 的 RWM 为 24/24，顺序 MALA 为 24/23，quasi-DEER 为 17/11，NUTS 为 24/9。点为各自有效集合的描述，共同集合上的损失比较另见完整附录。

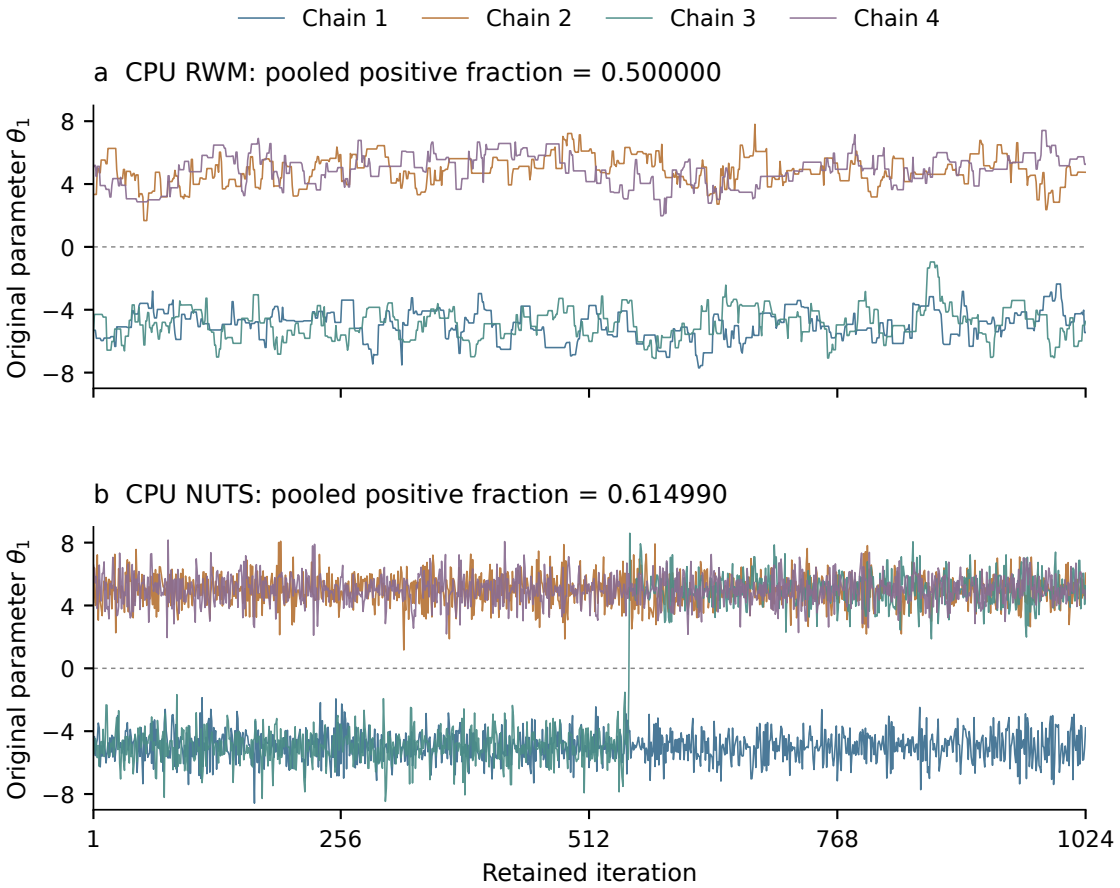


图 4: 双峰目标的保存路径。上图为 CPU 顺序 RWM，下图为完整 M1 审查中唯一出现符号穿越的 CPU NUTS 拟合，两者为同一原重复和 1024 步预算，初值相同但随机机制不同。绘出四链全部保留状态；θ_1 为原尺度上的第一参数。示例按上述事后规则选择，不据两份拟合比较性能。

径已有核验。该案例已参与开发，不作为未见模型族测试。新的数值包络给出 β 后验均值约 -0.62298325，0 米与 100 米处的换井概率均值约为 0.64703740 和 0.49590258。既有独立 MCMC 参考的 95% 等尾区间分别为 $[-0.8108, -0.4300]$、$[0.6197, 0.6733]$ 和 $[0.4652, 0.5262]$；这些分位数区间仍为有限样本估计，与均值的确定数值包络分别列示（补充材料第 2 节）。这是指定二维模型内的后验关联描述，非距离的因果效应。

表4比较 4096 步预算的所有工作流。CPU 顺序 RWM 的普通工作流耗时约 4.94 秒、α 参考平方差约 2.06×10^{-6}；CPU 顺序 MALA 约 10.58 秒、平方差约 0.58×10^{-6}；CPU NUTS 约 11.62 秒、平方差约 0.28×10^{-6}。同核时间并行执行没有改善这些轨迹的统计信息，却增加了本例的普通工作流耗时。新的确定数值包络使六个连续函数的参考半宽均不超过 10^{-8}。将这些区间传播到原配对损失差后，MALA/NUTS 的点排序保持不变；旧 MCMC 参考的 ± 2 MCSE 偏移虽可使排序翻转，但它不是实际求积误差界。数值参考误差在本例中已被限定，重复实验不确定性仍按原区间解释（补充材料第 2 节）。

工作流	普通耗时（秒）	α 参考平方差（10^{-6}）	β 参考平方差（10^{-6}）
CPU RWM seq	4.94	2.06 [1.16, 3.40]	4.91 [2.72, 7.94]
CPU Picard	11.59	2.06 [1.16, 3.40]	4.91 [2.72, 7.94]
CUDA RWM seq	5.73	2.06 [1.16, 3.40]	4.91 [2.72, 7.94]
CUDA Picard	6.01	2.06 [1.16, 3.40]	4.91 [2.72, 7.94]
CPU MALA seq	10.58	0.58 [0.39, 0.89]	1.11 [0.68, 2.00]
CPU quasi-DEER	68.85	0.58 [0.39, 0.89]	1.11 [0.68, 2.00]
CUDA MALA seq	13.99	0.58 [0.39, 0.89]	1.11 [0.68, 2.00]
CUDA quasi-DEER	45.51	0.58 [0.39, 0.89]	1.11 [0.68, 2.00]
CPU NUTS	11.62	0.28 [0.14, 0.51]	0.52 [0.32, 0.84]

表 4: 水井 W1 在每链 4096 次保留转移下的普通工作流耗时均值和参考平方差。每行 24 份原始四链重复；方括号为条件 95% BCa 区间。表中沿用冻结求积参考点；BCa 区间不传播参考误差，后续数值包络分析另列。耗时包含普通进程与 R 诊断，不含最外层 R 前端。其余五函数和 1024 步结果见完整结果附录图 R39–R45。

4096 步下，六个非稀有函数的保存 $\hat{R}$ 均不超过 1.01；但 $\mathbf{1}(\beta > 0)$ 在所有拟合中恒零，诊断仍未定。两规则包络的交集给出 $\Pr(\beta > 0) \in [4.7671124, 4.7673026] \times 10^{-11}$（向外舍入）。所有拟合的零事件估计因而具有 100% 的相对参考误差，尽管绝对平方差仅约 2.27×10^{-21}。这一结果明确了图 R45 中极小损失的含义：常规系数估计良好，并不意味着已经观测或可靠估计了罕见事件。

4.6 实际 R 调用的额外耗时

有限 Poisson 补测直接记录新 R 进程启动到首次样本和诊断可用的延迟，以及同会话再次完整调用的耗时。32 个进程、64 次调用均完成独立 NumPy 路径检查及 R 浮点字节核对，接受事件零失配。普通模式首次可用延迟的组中位数为 1.27–2.06 秒，同

会话再次调用为 0.014–0.340 秒。首次普通 MALA 顺序调用中还出现 5.22 秒观测，完整保留；四次技术重放的范围和耗时分解见补充材料。这项测量补足 R 前端的使用边界，与 Windows 正式比较分别报告；短 Poisson 轨迹没有据此获得精度认证。

5 讨论与结论

本文的主要发现是，时间并行执行的收益同时受核、设备和计时边界影响。在所测 torch 工作流中，CUDA Picard 的多数模型—预算组具有较低的普通工作流耗时点估计，而 quasi-DEER 的额外工作未被抵消；真实水井案例中的时间并行执行也没有减少普通调用成本。机制对照与这些结果相容：窗口扩大增加同时求值的位置，也可能增加重复映射、JVP 和等待确认的工作。当前探针尚不能唯一分离扫描、导数和主机控制各自的因果贡献。

这种限定与原方法的收益并不冲突。Picard 原文在 Apple M3 上对由时滞微分方程定义的昂贵黑箱似然报告约 2.52 倍墙钟收益 [2]，成本结构不同于本文多数廉价目标。本文的窗口与停止规则也属于适配实现。后续若要判断昂贵似然的适用边界，需要预先控制单次求值成本和批处理吞吐，而不是从现有网格中挑选最快的配置。

路径标准、函数误差和后验探索回答不同问题。H1 的函数均值差较小，却不能替代完整路径合格条件；M1 的合并符号比例可由均衡初值维持；L2 的绝对事件误差又可能掩盖相对误差。这些对照要求研究者结合所关心函数、初始化和误差尺度解释收益。W1 的数值包络收窄了参考不确定性，却不改变常量事件的探索局限。NUTS 补充定位表明，完整采样后诊断仍可使工作流因内存保护而终止；这些后处理费用与失败不能全归为转移核的性质。六个历史失败案例本轮未复现，原 105 项仍未定，因此保留成功条件比较和全部失败成本，不据点排序作无条件方法排名。

ParallelBayes 的贡献是可复用的目标/核/执行分离、固定已保存的随机数组的独立核验，以及从原始结果重建的成本和误差证据。Stan 仍仅接 CPU 顺序核，Pyro NUTS 是独立研究 CLI，R 前端耗时只在指定 Mac CPU 示例中直接测量。自动选择器、GPU NUTS 和 MEADS/LAPS 等更广的多链比较 [13, 14, 15] 不属于本版实测能力。现有结果支持按明确目标和计时边界使用该软件进行执行器研究；它们没有建立普遍推断加速或跨模型自动配置规则。

代码、数据与构建材料

源码、冻结协议、分析摘要和论文输入见ParallelBayes 研究分支，代码采用 MIT 许可。紧凑实验执行源固定为 `3a37a89`，Windows 交付源为 `94bebec`；H1 函数伴随分析重放旧输入，L2 重要性参考及 W1 数值包络采用独立新协议，三项补充与原主实验分开归档。CPU NUTS 有限定位绑定执行源 `1269690`，Windows 交付 `1a8759f`，已完成 Mac 独立接收；技术调用不增加正式统计重复。具体构建提交及输入 SHA256 随论文

包记录；新 H1/L2/W1 原始补充包目前保存在本地，汇总和重建脚本随研究分支提供。历史 CPU 原始证据位于cpu-review-v1 Release。Windows 紧凑原数组与 Mac 派生包及 NUTS 有限定位原件截至 2026 年 10 月 11 日存放于具有访问权限的草稿 Release，尚未公开；完整复现需要取得这些原件，不能仅凭公开源码下载完成。仓库的复现指南列出组件、清单及重建命令，本版修订记录列出可编译论文包和完整结果附录入口。

生成式工具用于辅助代码实现、分析组织、图表制作和中文表述。数值陈述与图表由保存的实际输入、独立参考和可重复构建检查支持；最终学术判断与稿件责任属于作者。

参考文献

[1] Zoltowski D.M., Wu S., Gonzalez X., et al. Parallelizing MCMC Across the Sequence Length. Advances in Neural Information Processing Systems, 2025, 38.

[2] Grazzi S., Zanella G. Parallel computations for Metropolis Markov chains with Picard maps. Biometrika, 2026, 113(2): asag022.

[3] Carpenter B., Gelman A., Hoffman M.D., et al. Stan: A Probabilistic Programming Language. Journal of Statistical Software, 2017, 76(1): 1–32.

[4] Roualdes E.A., Ward B., Carpenter B., Seyboldt A., Axen S.D. BridgeStan: Efficient in-memory access to the methods of a Stan model. Journal of Open Source Software, 2023, 8(87):5236.

[5] Cabezas A., Corenflos A., Lao J., Louf R., et al. BlackJAX: Composable Bayesian inference in JAX. arXiv:2402.10797, 2024.

[6] Bingham E., Chen J.P., Jankowiak M., et al. Pyro: Deep Universal Probabilistic Programming. Journal of Machine Learning Research, 2019, 20(28):1–6.

[7] NIMBLE development team. compareMCMCs: Tools for comparing MCMC efficiency from NIMBLE and other engines. 软件作者仓库，访问于 2026 年 10 月 3 日。

[8] Kulkarni S., Shah K.D., Arora N., et al. PPL Bench: Evaluation Framework For Probabilistic Programming Languages. arXiv:2010.08886v1, 2020. 软件来源：`https://github.com/facebookresearch/pplbench`。

[9] Magnusson M., Torgander J., Bürkner P.-C., Zhang L., Carpenter B., Vehtari A. posteriordb: Testing, Benchmarking and Developing Bayesian Inference Algorithms. Proceedings of Machine Learning Research, 2025, 258:1198–1206.

[10] Stan development team. posterior: Rhat, bulk ESS and tail ESS documentation. 软件文档，版本 1.7.0。

[11] Senne R. and contributors. ParallelMCMC.jl. 软件仓库，提交 7be9c86，访问于 2026 年 10 月 6 日。

[12] Sosa S. BayesForge: Bayesian Inference using numpyro and XLA. R 包 0.0.1 参考手册，访问于 2026 年 10 月 6 日。

[13] Hoffman M.D., Sountsov P. Tuning-Free Generalized Hamiltonian Monte Carlo. Proceedings of Machine Learning Research, 2022, 151:7799–7813.

[14] Robnik J., Seljak U. Faster Parallel MCMC: Metropolis Adjustment Is Best Served Warm. Proceedings of Machine Learning Research, 2026, 300:2719–2727.

[15] Dance H., Glaser P., Orbanz P., Adams R.P. Efficiently Vectorized MCMC on Modern Accelerators. Proceedings of Machine Learning Research, 2025, 267:12436–12458.

[16] Gonzalez X., Warrington A., Smith J.T.H., Linderman S.W. Towards Scalable and Stable Parallelization of Nonlinear RNNs. arXiv:2407.19115v3, 2025（初版 2024）.

[17] Vehtari A., Gelman A., Simpson D., Carpenter B., Bürkner P.-C. Rank-Normalization, Folding, and Localization: An Improved R-hat for Assessing Convergence of MCMC. Bayesian Analysis, 2021, 16(2):667–718.

[18] Modrák M., Moon A.H., Kim S., et al. Simulation-Based Calibration Checking for Bayesian Computation: The Choice of Test Quantities Shapes Sensitivity. Bayesian Analysis, 2025, 20(2):461–488. DOI:10.1214/23-BA1404.

[19] Owen A.B. Monte Carlo theory, methods and examples. 2013, Chapter 9: Importance sampling. 在线书稿。

[20] National Institute of Standards and Technology. Digital Library of Mathematical Functions, Section 3.5: Quadrature. Equations 3.5.6, 3.5.19–3.5.21，访问于 2026 年 10 月 10 日。

[21] Johansson F. Arb: Efficient Arbitrary-Precision Midpoint-Radius Interval Arithmetic. IEEE Transactions on Computers, 2017, 66(8):1281–1292.